SignCare AI

A Medical Communication Assistant for Deaf and Non Speaking Patients

Project Type: AI Based Mobile Application
Primary Platform: Android Mobile Application
Framework: Flutter
Backend: Python, Django REST Framework
AI and Computer Vision: MediaPipe, OpenCV, Machine Learning
Target Users: Deaf and non speaking patients, doctors, healthcare staff
Development Environment: Windows 11, Intel Core i5 10th Generation, 32 GB RAM, 512 GB SSD, Intel UHD Graphics
Document Version: 1.0
Date: October 2026

1. Executive Summary

SignCare AI is a proposed mobile application designed to improve communication between deaf and non speaking patients and healthcare professionals.

Many patients face difficulties explaining their pain, symptoms, emotions, medical concerns and other needs during a hospital visit. Traditional communication methods may not be sufficient when a patient cannot speak, cannot hear, is not comfortable writing, or does not have access to a sign language interpreter.

SignCare AI will provide multiple communication methods through one mobile application. These will include visual communication cards, body diagrams, symptom selection, text to speech, speech to text, doctor response options and, in later development phases, Pakistan Sign Language recognition.

The application will focus specifically on medical communication rather than general everyday communication.

The initial version will allow patients to select symptoms, indicate the location and intensity of pain, prepare a readable message and communicate it to a doctor. Doctors will be able to respond through text, speech and approved visual instructions.

Advanced versions may use computer vision and machine learning to recognize selected sign language gestures and convert them into text.

The main goal is to make medical communication more accessible, understandable and independent for deaf and non speaking patients.

2. Problem Statement

Deaf and non speaking patients may face communication barriers when visiting hospitals, clinics and emergency departments.

Patients may have difficulty communicating:

1. Where they feel pain.
2. How severe their pain is.

3. When their symptoms started.
4. Whether they have fever, dizziness, nausea or other symptoms.
5. What they need from the doctor.
6. Whether they understand medical instructions.
7. How they feel emotionally.
8. Whether they need immediate assistance.

Doctors may also face difficulties understanding a patient's signs, written messages or gestures, particularly when they are unfamiliar with the patient's communication method.

Existing general communication tools may not provide a complete medical workflow for selecting symptoms, describing pain, asking questions and understanding doctor instructions.

SignCare AI aims to address these problems through an accessible mobile interface designed specifically for healthcare communication.

3. Project Objectives

The main objectives are:

1. Develop an accessible Flutter mobile application for deaf and non speaking patients.
2. Enable communication between patients and doctors without requiring the patient to speak.
3. Convert typed messages into speech.
4. Convert a doctor's speech into readable text where supported.
5. Provide visual communication cards for common medical needs.
6. Allow patients to identify pain locations using a human body diagram.
7. Allow patients to select pain intensity, symptom type and symptom duration.
8. Support Urdu and English interfaces, with Roman Urdu text input.
9. Research Pakistan Sign Language recognition using computer vision.
10. Provide visual explanations and doctor approved medical instructions.
11. Support offline access to essential communication cards.
12. Protect patient privacy and avoid presenting AI predictions as confirmed medical facts.

4. Target Users

4.1 Deaf Patients

Patients who cannot hear spoken communication and need visual or written methods to communicate with doctors.

4.2 Non Speaking Patients

Patients who cannot speak or have difficulty producing understandable speech.

4.3 Patients with Multiple Communication Barriers

Patients who may need visual cards, text, gestures or alternative communication methods.

4.4 Doctors and Healthcare Staff

Healthcare professionals who need to understand patient concerns and communicate instructions clearly.

4.5 Caregivers and Interpreters

People who may assist patients when requested by the patient.

The application should not assume that every deaf person is non speaking or that every non speaking person uses sign language. Users should be able to select whichever communication methods work for them.

5. Complete Feature List

The proposed features are divided into three development levels.

5.1 Level One: Minimum Viable Product

These features should be developed first.

Feature 1: Patient Mode

The application will provide a simple patient interface with large buttons and clear labels.

Main options:

1. Describe My Symptoms.
2. Communication Cards.
3. Type a Message.
4. Speak My Message.
5. Doctor's Response.
6. Language Settings.

The interface should be simple enough for a new user to understand without extensive instructions.

Feature 2: Text to Speech

Patients who cannot speak will be able to type a message and play it aloud.

Example:

Patient types: "I have pain in my stomach."

The application reads the message aloud so that the doctor can hear it.

Additional functions:

1. Play message.
2. Repeat message.
3. Stop playback.
4. Edit text.
5. Select supported voice or language.

Recommended technology: Android TextToSpeech or a compatible Flutter package.

Urdu voice quality and device support must be tested before implementation is finalized.

Feature 3: Speech to Text

The doctor can speak, and the application can display the recognized words on the screen.

Example:

Doctor says: "When did your pain start?"

The application displays the recognized sentence so the patient can read it.

Additional functions:

1. Start listening.
2. Stop listening.
3. Display recognized text.
4. Repeat the recognized text.
5. Allow manual correction or typing if recognition fails.

Recommended technology: Android speech recognition or a tested speech recognition service.

Speech recognition accuracy depends on language, background noise, microphone quality and device support. Urdu and English should be tested separately.

Feature 4: Visual Communication Cards

The application will contain cards with simple illustrations, text labels and optional speech playback.

Suggested categories:

Basic Needs

1. I need help.
2. I need water.
3. I need to use the washroom.
4. I feel uncomfortable.
5. I need someone to explain again.

Pain and Symptoms

1. I have pain.
2. I have a headache.
3. I have stomach pain.
4. I have chest pain.
5. I have a fever.
6. I feel dizzy.
7. I feel nauseated.
8. I have difficulty breathing.
9. I feel weak.
10. I have an injury.

Medical Communication

1. Please explain slowly.
2. I do not understand.
3. Please write it down.
4. I need to ask a question.
5. I need more information.

6. I want to contact my caregiver.
7. Please repeat the instructions.

Emotional and Comfort Cards

1. I feel scared.
2. I feel worried.
3. I feel uncomfortable.
4. I feel calm.
5. I need reassurance.
6. I need privacy.

These cards should communicate what the patient selects. They should not automatically diagnose a condition.

Feature 5: Interactive Human Body Diagram

The application will display front and back views of a human body.

Patients will tap the relevant area to identify where they feel pain or discomfort.

Suggested body regions:

1. Head.
2. Eyes.
3. Ears.
4. Neck.
5. Chest.
6. Abdomen.
7. Back.
8. Arms.
9. Hands.
10. Legs.
11. Knees.
12. Feet.

The application will highlight the selected region and allow the patient to continue to the symptom details screen.

The diagram is a communication aid, not a diagnostic tool.

Feature 6: Pain Intensity Selection

Patients will be able to select the intensity of their pain.

Suggested scale:

0: No pain.

1 to 3: Mild pain.

4 to 6: Moderate pain.

7 to 9: Severe pain.

10: Worst pain imaginable.

The scale should be accompanied by simple visual indicators and text. The patient must choose the rating personally.

Feature 7: Symptom Duration

Patients will select when their symptoms started.

Suggested options:

1. Just now.
2. A few minutes ago.
3. A few hours ago.
4. Today.
5. Yesterday.
6. Several days ago.
7. I am not sure.

Additional options can record whether the symptom is constant or comes and goes.

Feature 8: Symptom Description

Patients will be able to select how a symptom feels.

For pain, suggested options include:

1. Sharp.
2. Dull.
3. Burning.
4. Throbbing.
5. Cramping.
6. Pressure.
7. Other.
8. Not sure.

These options should be optional. Patients should not be forced to choose a description they cannot identify.

Feature 9: Symptom Summary

After selecting their symptoms, patients will see a summary before sharing it with the doctor.

Example:

"My stomach hurts. The pain is moderate. It started a few hours ago. I also feel nauseated."

The summary must be generated only from information selected or entered by the patient.

The patient should be able to review, correct or remove any item before showing the summary.

Feature 10: Doctor Response Screen

Doctors will be able to communicate with patients using readable text and supported speech features.

The patient will be able to:

1. Read the doctor's question.
2. Replay spoken instructions.
3. Select a visual answer.
4. Type a response.
5. Ask the doctor to repeat or simplify an explanation.

Feature 11: Urdu and English Interface

The application should initially support Urdu and English interface labels.

Roman Urdu text input should also be supported.

Examples:

Urdu: "Mujhe pait mein dard hai."

English: "I have stomach pain."

Roman Urdu should be treated as a text entry option. It should not be assumed that speech recognition or text to speech will automatically support Roman Urdu.

Feature 12: Offline Communication Cards

Essential communication cards, body diagrams and basic symptom selection should work without internet access.

Speech recognition, cloud based AI and some text to speech voices may still require network access or device specific support.

The application should clearly indicate when an online feature is unavailable.

5.2 Level Two: Advanced Features

These features should be added after the MVP is working reliably.

Feature 13: Pakistan Sign Language Recognition

The application will use the phone camera to recognize selected Pakistan Sign Language gestures.

The initial goal should be a small vocabulary of medically useful signs.

Potential categories:

1. Pain.
2. Help.
3. Water.
4. Doctor.
5. Hospital.

6. Medicine.
7. Fever.
8. Head.
9. Stomach.
10. Stop.
11. Yes.
12. No.

The final vocabulary must be selected with fluent Pakistan Sign Language users and healthcare professionals.

The system will process the camera input, extract hand landmarks and classify supported signs.

Recognized signs will appear as text. The patient will be asked to confirm the result before it is spoken or added to the consultation summary.

If confidence is low, the application should display "Sign not recognized" and allow manual selection.

Important: a complete sign language is not simply a collection of hand gestures. Facial expressions, body movement, spatial relationships, grammar and context can all be important. A limited gesture classifier must not be advertised as a complete Pakistan Sign Language translator.

Feature 14: Visual Doctor Questions

The application will show common questions with simple visual answer options.

Examples:

Question: "Are you feeling pain?"

Options: Yes, No, Not sure.

Question: "When did it start?"

Options: Today, Yesterday, Several days ago, Not sure.

Question: "Is the pain getting worse?"

Options: Yes, No, About the same, Not sure.

The doctor or clinician must be able to review the responses in context.

Feature 15: Visual Medical Instructions

Doctors will be able to provide simple instructions through approved text and pictograms.

Examples:

1. Sit down.
2. Take a deep breath, when clinically appropriate.
3. Wait here.
4. Follow the confirmed prescription.
5. Return for the scheduled appointment.
6. Ask for assistance if the condition worsens.

Instructions should be written or approved by qualified healthcare professionals. The AI system must not independently prescribe medicines or invent treatment instructions.

Feature 16: AI Assisted Consultation Summary

An AI service may help organize patient selected symptoms into a concise summary.

Example input:

Body area: Head.

Symptom: Pain.

Intensity: Moderate.

Duration: Since morning.

Output:

"The patient selected headache, described the intensity as moderate and reported that it started this morning."

The system must preserve the source of each detail and must not invent a diagnosis.

For the first version, a rule based template is preferable to a generative AI model because it is easier to test and control.

Feature 17: Limited Offline Sign Recognition

Once a sign recognition model has been trained and tested, a lightweight version may be integrated into the mobile application.

This may allow supported signs to be recognized without an internet connection.

The model must be small enough to run acceptably on the target phone.

5.3 Level Three: Future Features

These features are optional and should not delay the initial release.

1. Secure patient profiles.
2. Consultation history.
3. Clinician confirmed medical instructions.
4. Medication reminders based on confirmed prescriptions.
5. Emergency staff alert functionality.
6. Additional Pakistani languages.
7. Expanded Pakistan Sign Language vocabulary.
8. Support for additional signers and regional variations.
9. Hospital system integration.
10. Optional caregiver access with patient consent.
11. Accessibility settings for text size, contrast and preferred communication mode.
12. Sound recognition for selected environmental alerts.

Facial emotion estimation should remain optional research. The application should prefer patient selected emotion cards because facial expressions cannot reliably establish what someone is feeling.

6. Comparison with SpeakLiz

SpeakLiz is an existing accessibility application for deaf and hard of hearing users.

Its public app listings describe the following capabilities:

1. Speech to text.
2. Text to speech.
3. Sign language recognition with stated limitations.
4. Environmental sound recognition.
5. Recognition of various surrounding sounds and alerts.
6. Support for multiple languages, subject to feature specific availability.

The current capabilities should be checked directly in the installed version before using this comparison in a formal research report.

6.1 Feature Comparison Table

Feature	SpeakLiz	Proposed SignCare AI
Speech to text	Listed	Included
Text to speech	Listed	Included
Sign language recognition	Listed, limited ASL support described	PSL focused research
Environmental sound recognition	Listed	Optional
Emergency sound alerts	Sound recognition is listed	Optional future feature
Medical communication cards	Not confirmed from public listing	Core feature
Interactive body diagram	Not confirmed	Core feature
Pain intensity selection	Not confirmed	Core feature
Symptom duration	Not confirmed	Core feature
Symptom description	Not confirmed	Core feature
Patient selected emotion cards	Not confirmed	Planned
Doctor question and answer workflow	Not confirmed	Planned
Visual medical instructions	Not confirmed	Planned
Patient reviewed symptom summary	Not confirmed	Planned
Urdu medical interface	Must verify current support	Design goal
Roman Urdu text input	Not confirmed	Planned
Offline medical cards	Must verify current behavior	Included in MVP
Medical consultation workflow	Not confirmed	Main focus
Doctor approved instructions	Not confirmed	Planned
Medical records	Not confirmed	Optional future feature

"Not confirmed" means that the feature was not established by the public listing used for this comparison. It does not prove that the feature is absent.

6.2 Main Differentiation

SignCare AI should focus on a specific problem: communication between deaf and non speaking patients and healthcare professionals.

Its potential differentiators are:

1. Medical symptom selection.
2. Pain location and intensity reporting.
3. Pakistan Sign Language research.
4. Urdu and Roman Urdu accessibility.
5. Two way communication between patients and doctors.
6. Visual questions and responses.
7. Patient reviewed consultation summaries.
8. Doctor approved visual instructions.
9. Offline access to essential communication tools.

The goal is not simply to provide more features than SpeakLiz. The goal is to make a medical visit easier and safer for the intended users.

7. Recommended Technology Stack

7.1 Mobile Application

Flutter

Purpose:

1. Build the Android application.
2. Design accessible user interfaces.
3. Create communication cards and body diagrams.
4. Integrate camera and microphone access.
5. Implement navigation and local settings.
6. Integrate speech and machine learning features.

Programming language: Dart.

Recommended editor: Visual Studio Code.

Android Studio may be installed to manage the Android SDK and build tools.

7.2 Backend

Python and Django REST Framework

Purpose:

1. Provide APIs for the mobile application.
2. Store consultation information when required.
3. Manage user authentication.
4. Apply access controls.
5. Process structured symptom information.
6. Support future clinician dashboards and integrations.

The backend should be introduced after the basic local mobile workflow is stable.

7.3 Database

SQLite for early prototypes

SQLite is sufficient for local testing and small prototype data.

PostgreSQL for deployed backend services

PostgreSQL is preferable when multiple users, concurrent requests and production data management are required.

Only collect and store data that the application genuinely needs.

7.4 Computer Vision

OpenCV

Purpose:

1. Access and process camera frames.
2. Resize and prepare images.
3. Support image processing and model input.
4. Help build the gesture recognition prototype.

MediaPipe

Purpose:

1. Detect hands.
2. Extract hand landmarks.
3. Track hand positions.
4. Prepare landmark data for a gesture classifier.

The initial approach should classify a limited vocabulary rather than attempt full sign language translation.

7.5 Machine Learning

Recommended initial tools:

1. Python.
2. NumPy.
3. Pandas.
4. Scikit learn.
5. MediaPipe.
6. TensorFlow or another compatible framework if a neural model is required.

Start with a simple landmark based classifier. Only move to a larger neural network if testing shows that the simpler approach is insufficient.

7.6 Speech Tools

For text to speech, begin with Android's native TextToSpeech capability.

For speech to text, begin with Android speech recognition or a compatible service.

Test English and Urdu independently. Do not assume every device provides the same offline recognition or voice capabilities.

7.7 AI Summary

Begin with a rule based summary generator.

A generative AI API can be evaluated later if it provides a clear benefit. Any AI generated summary must be editable and limited to the information provided by the patient or clinician.

7.8 Version Control

Git

Purpose:

1. Track source code changes.
2. Create checkpoints before major modifications.
3. Restore earlier working versions.
4. Back up the project to a private remote repository.

Never upload passwords, API keys, patient records or private recordings to a public repository.

8. Laptop Capability and Development Environment

Target laptop specifications:

Processor: Intel Core i5 10th Generation.

Memory: 32 GB RAM.

Storage: 512 GB SSD.

Graphics: Intel UHD integrated graphics.

Operating system: Windows 11.

8.1 Tasks That Should Work Well

1. Flutter application development.
2. Django backend development.
3. SQLite and PostgreSQL development.
4. OpenCV image processing.
5. MediaPipe hand landmark extraction.
6. Small machine learning models.
7. Static gesture classification.
8. Basic dynamic gesture experiments.
9. API integration.
10. UI testing and Android application builds.

8.2 Tasks That May Be Limited

1. Training large neural networks from scratch.

2. Training large vision models.
3. Training large language models.
4. Processing very large video datasets.
5. Running multiple heavy emulators and machine learning workloads together.

The Intel UHD graphics is integrated rather than a dedicated high performance GPU. Development should therefore focus on lightweight models and pretrained components.

Cloud GPU services can be used later if model training becomes too slow.

8.3 Recommended Testing Device

Use a physical Android phone for routine testing whenever possible.

This reduces emulator overhead and allows realistic testing of:

1. Camera performance.
2. Microphone input.
3. Speech playback.
4. Gesture recognition.
5. Touch interaction.
6. Performance and battery usage.

Keep sufficient SSD space available for Android tools, Flutter builds, datasets and project backups.

9. Application Architecture

The application will use three main layers.

9.1 Layer One: Flutter Mobile Application

Responsibilities:

1. Patient interface.
2. Symptom selection.
3. Communication cards.
4. Body diagram.
5. Text input and speech output.
6. Doctor response display.
7. Camera interface for supported sign recognition.
8. Local preferences and offline assets.

9.2 Layer Two: Django REST API

Responsibilities:

1. Receive and validate requests.
2. Manage authenticated users if accounts are required.
3. Store approved consultation information.
4. Retrieve consultation data.
5. Apply authorization rules.
6. Support future clinical integrations.

9.3 Layer Three: AI and Machine Learning

Responsibilities:

1. Extract hand landmarks.
2. Classify supported signs.
3. Organize patient selected symptoms.
4. Prepare structured communication output.
5. Return a prediction only when the model has adequate confidence.

A typical request flow will be:

Patient selects symptoms in Flutter.

The application creates a structured summary.

The patient reviews the summary.

The summary is displayed to the doctor.

If backend storage is required, the application sends the approved information to the Django API.

The backend validates the request and stores only the permitted data.

Sign recognition will follow a separate pipeline and should not automatically convert uncertain predictions into confirmed medical information.

10. Proposed Application Screens

The first version should contain the following screens.

Screen 1: Welcome Screen

Application name and a short explanation.

Button: Start.

Screen 2: Language Selection

Options:

1. Urdu.
2. English.

Roman Urdu text entry should be available where appropriate.

Screen 3: Patient Home Screen

Main options:

1. Describe Symptoms.
2. Communication Cards.

3. Type a Message.
4. Supported Sign Recognition.
5. Doctor's Response.
6. Settings.

Screen 4: Body Diagram

Front and back body views.

The patient selects the affected area.

Screen 5: Symptom Details

The patient selects:

1. Symptom type.
2. Pain intensity.
3. Duration.
4. Additional details.

Screen 6: Communication Cards

Cards are grouped by basic needs, symptoms, medical questions and feelings.

Selecting a card displays the message and optionally speaks it.

Screen 7: Message Preview

Displays the patient's selected information in a readable format.

The patient can edit or remove information.

Screen 8: Speech and Text

Allows typed messages to be spoken aloud.

Also provides speech recognition for doctor responses.

Screen 9: Sign Recognition

Displays camera input and supported sign predictions.

The patient must confirm the result.

An unrecognized sign can be entered manually or selected from communication cards.

Screen 10: Doctor Response

Displays the doctor's text and approved visual instructions.

The patient can request repetition or select a response option.

Screen 11: Settings

Options for:

1. Language.
2. Text size.
3. Preferred communication mode.
4. Speech settings.
5. Privacy settings.

Screen 12: Consultation History

This should be added only when secure storage, consent, access control and deletion requirements have been designed and implemented.

11. Database Design

The initial prototype should avoid storing unnecessary personal or medical information.

11.1 User Profile

Suggested fields:

1. User ID.
2. Preferred language.
3. Preferred communication mode.
4. Accessibility preferences.
5. Created date.

11.2 Consultation

Suggested fields:

1. Consultation ID.
2. Created date.
3. Status.
4. Consent information, where applicable.
5. Optional authorized user reference.

11.3 Symptom Item

Suggested fields:

1. Symptom item ID.
2. Consultation ID.
3. Body region.
4. Symptom type.
5. Pain intensity.
6. Duration.
7. Additional patient entered details.

11.4 Message

Suggested fields:

1. Message ID.
2. Consultation ID.
3. Sender role.
4. Message text.
5. Creation time.
6. Source, such as patient entry or clinician entry.

11.5 Doctor Instruction

Suggested fields:

1. Instruction ID.
2. Consultation ID.
3. Instruction text.
4. Associated visual asset.
5. Clinician confirmation status.
6. Creation time.

11.6 Sign Recognition Sample Metadata

Suggested fields:

1. Sample ID.
2. Sign label.
3. Signer consent reference.
4. Recording session.
5. Data split.
6. Annotation status.

Sign language recordings may contain identifiable information. Their collection and storage require specific consent and access restrictions.

12. Project Folder Structure

A suggested project structure is:

signcare_ai/

mobile_app/

lib/

main.dart

app/

features/

patient_home/

symptom_selector/

communication_cards/

speech_tools/

sign_recognition/

doctor_response/

shared/

assets/

icons/

illustrations/

cards/

backend/

manage.py

config/

apps/

users/

consultations/

messages/

ml/

data_docs/

notebooks/

preprocessing/

training/

evaluation/

models/

docs/

requirements.md

privacy_and_consent.md

test_plan.md

README.md

This structure separates mobile development, backend services, machine learning experiments and documentation.

13. Complete Development Roadmap

The project should be developed in a planned sequence. Each phase should be tested before moving to the next.

Phase 1: Problem Validation

Estimated duration: 3 to 5 days.

Tasks:

1. Discuss the idea with deaf and non speaking people.
2. Consult at least one healthcare professional.
3. Consult a fluent Pakistan Sign Language user or qualified interpreter.
4. Identify the most common communication problems during medical visits.
5. Select the first set of medical phrases and symptom categories.
6. Confirm which features are necessary for the first version.
7. Write down privacy, consent and accessibility requirements.

Deliverable: A validated problem statement and approved initial feature list.

Phase 2: Development Environment Setup

Estimated duration: 1 to 2 days.

Install:

1. Git.
2. Visual Studio Code.
3. Flutter SDK.
4. Dart SDK through Flutter.
5. Android SDK and platform tools.
6. Python.
7. A Python virtual environment.
8. Django and Django REST Framework.

Configure the environment variables and Android SDK paths as required by the official documentation.

Run:

flutter doctor

Resolve the required setup issues.

Enable Developer Options and USB Debugging on the Android phone.

Connect the phone to the laptop and verify that Flutter can detect it.

Create a test project:

flutter create signcare_mobile

Open the project in Visual Studio Code.

Run:

flutter run

Confirm that the starter application works on the physical phone.

Deliverable: A working Flutter development environment.

Phase 3: User Interface Design

Estimated duration: 3 to 5 days.

Tasks:

1. Sketch all initial screens.
2. Define the navigation flow.
3. Design the body diagram.
4. Prepare communication card illustrations.
5. Select readable fonts and accessible colors.
6. Add large touch targets.
7. Use text labels alongside icons.
8. Prepare Urdu and English interface text.
9. Test the screens with intended users.

Deliverable: An approved screen design and navigation plan.

Phase 4: Flutter MVP Development

Estimated duration: 1 to 2 weeks.

Tasks:

1. Implement the welcome screen.
2. Implement language selection.
3. Build the patient home screen.
4. Implement communication cards.
5. Add the interactive body diagram.
6. Add symptom type selection.
7. Add pain intensity selection.
8. Add symptom duration.
9. Generate an editable symptom summary.
10. Integrate text to speech.
11. Integrate speech to text.
12. Add manual input fallbacks.
13. Make essential cards available offline.
14. Test the application on the physical phone.

Do not begin complex sign language model training during this phase.

Deliverable: A working application that supports basic medical communication.

Phase 5: Backend Development

Estimated duration: 4 to 7 days for a basic prototype.

Create a Python virtual environment.

Install Django and Django REST Framework.

Create the Django project and applications.

Define the required data models.

Create API endpoints for the information that genuinely needs to be shared or stored.

Validate incoming requests.

Add appropriate authentication and permissions before handling sensitive information.

Connect Flutter to the local API.

Test the complete request and response flow.

Use synthetic data during early development.

Deliverable: A functioning Flutter application connected to a Django backend.

Phase 6: Pakistan Sign Language Recognition Research

Estimated duration: Several weeks for an initial prototype. Reliable recognition may take considerably longer.

Tasks:

1. Consult fluent Pakistan Sign Language users.
2. Select a small set of useful medical signs.
3. Define each sign and its intended meaning.
4. Obtain informed consent before recording samples.
5. Record examples from multiple consenting signers.
6. Label and organize the samples.
7. Extract hand landmarks with MediaPipe.
8. Prepare the landmark dataset.
9. Train a simple baseline classifier.
10. Evaluate the model on signers not used during training.
11. Measure errors and confusion between signs.
12. Integrate the prototype into Flutter only after initial testing.

The first model should aim to recognize approximately 10 to 20 carefully selected signs. This is a research target, not a guaranteed result.

Deliverable: A limited vocabulary sign recognition prototype.

Phase 7: AI and Summary Features

Estimated duration: 3 to 7 days for an initial controlled implementation.

Tasks:

1. Define the allowed symptom fields.
2. Generate summaries from structured patient selections.
3. Preserve the original selected values.
4. Allow the patient to review and edit the summary.
5. Test for missing or incorrect information.
6. Evaluate an AI API only if it offers a clear improvement.
7. Prevent the system from inventing diagnoses or treatment advice.

Deliverable: A controlled symptom summary feature.

Phase 8: Testing and Improvements

Estimated duration: 1 to 2 weeks.

Test:

1. Basic communication flow.
2. Urdu and English text display.
3. Speech output.
4. Speech recognition in quiet and noisy environments.
5. Body region selection.
6. Pain intensity and duration selection.
7. Offline communication cards.
8. API connectivity.
9. Permission handling.
10. Sign recognition across different users and lighting conditions.
11. Unrecognized sign handling.
12. Accessibility and ease of use.

Collect feedback and fix the most important problems.

Deliverable: A tested prototype with documented limitations.

Phase 9: Pilot and Deployment Preparation

Estimated duration: Depends on testing and organizational requirements.

Tasks:

1. Review the application with target users.
2. Obtain clinical and accessibility feedback.
3. Review privacy and security controls.
4. Prepare a privacy notice.
5. Define consent, retention and deletion rules.
6. Test the final Android build.
7. Prepare application documentation.
8. Conduct a supervised pilot where appropriate.
9. Resolve issues before wider release.

Deliverable: A prototype ready for a controlled pilot, subject to appropriate approvals.

14. Suggested Eight Week Schedule

Week 1: Problem validation, requirements and screen sketches.

Week 2: Flutter installation, development environment and phone testing.

Week 3: Navigation, communication cards and body diagram.

Week 4: Speech features, symptom summary and offline support.

Week 5: Django API, database design and integration.

Week 6: Pakistan Sign Language vocabulary selection and dataset preparation.

Week 7: MediaPipe landmark extraction and a small gesture classifier.

Week 8: Testing, corrections, documentation and demonstration.

This is an approximate schedule for a solo developer. If the sign language dataset is not ready, prioritize a complete and tested medical communication MVP rather than rushing an unreliable recognition model.

15. Testing and Evaluation Criteria

The project should be evaluated using measurable outcomes.

Usability

Can a new user select a symptom and communicate it without unnecessary assistance?

Communication Accuracy

Does the generated summary accurately represent what the patient selected?

Speech Recognition

How often does the speech recognition system correctly transcribe the tested language in different environments?

Sign Recognition

Measure accuracy, precision, recall and a confusion matrix for each supported sign.

Test with signers excluded from the training dataset.

Accessibility

Are the buttons readable, easy to tap and understandable without relying only on color?

Offline Functionality

Can the user access essential cards and symptom selection without an internet connection?

Reliability

Does the application recover gracefully when the network, camera or microphone is unavailable?

Privacy

Are permissions, access controls and data deletion behavior working correctly?

Do not claim clinical effectiveness or sign language translation accuracy without suitable evidence.

16. Medical Safety and Privacy Requirements

1. The application must not replace a qualified clinician or sign language interpreter.
2. The application must not diagnose a medical condition independently.
3. Patient selected information must remain distinguishable from AI generated content.
4. AI summaries must not invent symptoms, diagnoses, medication names or doses.
5. The patient should review information before it is shared.
6. Doctor instructions must be confirmed by the responsible healthcare professional.
7. Facial expressions must not be treated as reliable proof of emotion.
8. Emergency guidance must not depend on an unverified AI prediction.
9. The application must not claim an emergency alert was delivered unless delivery is confirmed.
10. Sign language recordings must be collected with informed consent.
11. Personal and medical data must be protected with suitable access controls.
12. Data collection must be limited to what is necessary.
13. Synthetic data should be used during early development.
14. The project must undergo appropriate clinical, privacy and security review before real world medical deployment.

17. Main Risks and Solutions

Risk: Insufficient Pakistan Sign Language data

Solution: Start with a small, expert reviewed vocabulary and collect consented examples from multiple signers.

Risk: Model performs well only for the people in its training data

Solution: Keep entire signers out of the training set when evaluating generalization.

Risk: Urdu speech recognition performs poorly

Solution: Test early and retain manual text input and visual communication cards as fallbacks.

Risk: AI creates an inaccurate summary

Solution: Use structured templates, preserve the original patient selections and require user review.

Risk: The project becomes too large

Solution: Freeze the MVP feature list and postpone advanced features until the core communication workflow works.

Risk: Sensitive information is exposed

Solution: Minimize stored data, use access controls and avoid real patient records during development.

18. Final Recommended Feature Scope

The first release should contain:

1. Flutter patient interface.
2. Urdu and English interface options.
3. Roman Urdu text entry.
4. Medical communication cards.
5. Interactive human body diagram.
6. Pain intensity and symptom duration selection.
7. Editable symptom summary.
8. Text to speech.
9. Speech to text with manual fallback.
10. Basic doctor response interface.
11. Offline access to essential cards.
12. A clear privacy and accessibility design.

The second release should focus on:

1. Django backend integration.
2. Secure consultation storage where required.
3. Visual doctor questions.
4. Doctor approved visual instructions.
5. AI assisted summary improvements.
6. Limited Pakistan Sign Language recognition.

The third release may include:

1. Expanded sign vocabulary.
2. Improved offline sign recognition.
3. Secure consultation history.
4. Optional reminders.
5. Hospital integrations.
6. Additional language support.

19. Final Development Recommendation

The project should be developed in the following order:

First, validate the medical communication problem with intended users.

Second, finalize the MVP requirements and design the screens.

Third, configure Flutter and test the application on a physical Android phone.

Fourth, build the communication cards, body diagram, symptom selection and message preview.

Fifth, integrate speech output and speech recognition with manual fallbacks.

Sixth, connect the Django backend when the local communication workflow is stable.

Seventh, begin limited Pakistan Sign Language recognition research using MediaPipe and a properly prepared dataset.

Eighth, evaluate the model on previously unseen signers and document its limitations.

Ninth, conduct usability, privacy and safety testing before a supervised pilot.

The laptop specifications are sufficient for Flutter development, Django, OpenCV, MediaPipe and small machine learning experiments. Large scale model training may require cloud computing.

The biggest technical challenge is not building the app interface. It is obtaining suitable Pakistan Sign Language data and building a model that recognizes signs reliably across different users and real world conditions.

A successful first version should prioritize reliable communication over the number of AI features.

20. Official Technical References

Flutter documentation: https://docs.flutter.dev/

Android developer documentation: https://developer.android.com/

Django documentation: https://docs.djangoproject.com/

Django REST Framework: https://www.django-rest-framework.org/

MediaPipe documentation: https://ai.google.dev/edge/mediapipe/

OpenCV documentation: https://docs.opencv.org/

SpeakLiz on Google Play: https://play.google.com/store/apps/details?id=com.talovstudio.speakliz

SpeakLiz on Apple App Store: https://apps.apple.com/pk/app/speakliz-for-deaf-people/id1239857536

These references should be checked during implementation because software versions, supported features and platform requirements may change.

End of Document